COMPUTER LABORATORY • VIVA PREPARATION

Random Forest Car Evaluation

A simple, detailed guide to your Assignment 9(A) Jupyter Notebook

Experiment	Ensemble Learning – Random Forest Classifier
Task	Predict a car evaluation category from its features
Tools	Python • pandas • scikit-learn
Model setup	One-Hot Encoding + 100-tree Random Forest pipeline

What this guide covers

Line-by-line code explanation; theory of Decision Trees, Random Forest and Ensemble Learning; every imported function and parameter; preprocessing; train/test split; evaluation metrics; viva questions; tricky cross-questions; and possible improvements.

Accuracy values are not invented: your notebook reads Car_Evaluation.csv, but that CSV was not among the uploaded files. Run the notebook to record the actual results.

Prepared from the uploaded Assignment_9(A).ipynb and lab assignment sheet.

1. Experiment and dataset

Problem: Train a model that predicts the evaluation class of a car using information such as purchase price, maintenance cost, number of doors, passenger capacity, luggage boot size and safety level. This is the Random Forest option under Ensemble Learning in your lab assignment.

Classification or regression?

This is classification because the output is a category/label (for example, unacc, acc, good or vgood), not a continuous numeric value. Regression would be used to predict a number, such as a car price.

Typical columns in the standard Car Evaluation dataset

Column	Meaning	Typical values
buying	Buying price level	vhigh, high, med, low
maint	Maintenance cost	vhigh, high, med, low
doors	Number of doors	2, 3, 4, 5more
persons	Passenger capacity	2, 4, more
lug_boot	Luggage boot size	small, med, big
safety	Safety level	low, med, high
class	Target evaluation class	unacc, acc, good, vgood

These are the usual fields in the public dataset. Your notebook does not print the column names explicitly, so confirm them using cars.columns and inspect cars.head() when running your own CSV.

What is a row?

One row represents one car configuration described by the feature values and its known evaluation class.

What is a feature?

An input variable used to make the prediction. In this task, buying, maint, doors, persons, lug_boot and safety are typical features.

What is the target?

The output the model must learn to predict. Here it is the class column.

2. Imports: libraries and functions

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import make_pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Import	Purpose in this notebook
pandas (pd)	Loads and manages tabular data. read_csv reads the CSV into a DataFrame.
NumPy (np)	Numerical arrays and calculations. Imported here but not used in the shown code.
train_test_split	Divides features and labels into training and testing portions.
OneHotEncoder	Converts category/text features into numeric indicator columns.
make_pipeline	Connects preprocessing and the classifier into one workflow.
RandomForestClassifier	Creates the ensemble classifier made of multiple decision trees.
accuracy_score	Calculates the fraction of correct predictions.
confusion_matrix	Compares actual classes with predicted classes.
classification_report	Prints precision, recall, F1-score and support for each class.

What is scikit-learn?

A Python machine-learning library. It provides algorithms, preprocessing methods, model-selection utilities, pipelines and evaluation metrics.

Why is NumPy imported if it is unused?

It may be a leftover or a common starter import. It can be removed from this notebook without changing the current results.

What is a DataFrame?

A two-dimensional table with labelled rows and columns, similar to a spreadsheet.

3. Code walkthrough: load, split, prepare

Cell 1 – Load and inspect the CSV

```
cars = pd.read_csv('Car_Evaluation.csv')
cars.head()
```

pd.read_csv() reads a comma-separated file and returns a pandas DataFrame. The file must be in the current working directory, or the code must use its correct path. **cars.head()** displays the first five rows by default. It is an inspection step and does not change the data.

Cell 2 – Separate features and target

```
X = cars.drop(columns='class')
y = cars['class']
```

X is the input feature table. **drop(columns=...)** returns the DataFrame without the target column. **y** is the target Series selected from the class column. The model learns a mapping from X to y.

Why remove class from X?

If the target column remains among the inputs, the model is given the answer while learning. This is called target leakage; it can make evaluation misleadingly high.

Cell 3 – Split the dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

X_train/y_train: data used to learn. **X_test/y_test:** held-out data used to evaluate. test_size=0.2 reserves 20% for testing and 80% for training. random_state=42 makes the random split reproducible. stratify=y tries to preserve class proportions in both sets.

Why not test on training data?

The model has already seen training rows. Testing on unseen rows gives a more honest estimate of how it may perform on new examples.

4. Code walkthrough: encoding and training

Cell 4 – Pipeline, fit and predict

```
model = make_pipeline(
    OneHotEncoder(handle_unknown='ignore'),
    RandomForestClassifier(n_estimators=100, random_state=42)
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Why encoding is needed

Features such as “high”, “low”, “small” and “big” are categorical text. Most scikit-learn estimators require numeric inputs. One-Hot Encoding creates a separate 0/1 column for each category. For example, safety=medium among low/medium/high becomes [0, 1, 0].

What does handle_unknown="ignore" do?

If a test or future row contains a category not seen when the encoder was fitted, the encoder ignores that unknown category instead of raising an error. It handles unseen input categories, not unseen target classes.

Random Forest parameters

n_estimators=100 means the forest has 100 decision trees. More trees can stabilize predictions but increase computation. **random_state=42** makes randomized choices repeatable; it does not guarantee better accuracy.

What does make_pipeline do?

It chains the encoder and classifier. During fit, it learns categories from training features, transforms them, and trains the forest. During predict, it applies the same learned encoding before producing predictions. This keeps training and inference preprocessing consistent.

What do fit and predict mean?

fit(X_train, y_train) learns from the training examples. **predict(X_test)** returns one predicted class for each test row. The output is stored in predictions.

5. Theory: Decision Trees and Random Forest

Decision Tree

A decision tree asks a sequence of feature-based questions. Each internal node is a test, each branch is an outcome, and each leaf gives a prediction. A single deep tree can learn the training data too closely and overfit.

Ensemble Learning

Ensemble learning combines multiple models to produce a stronger or more stable prediction. Common approaches include bagging, boosting and stacking. Random Forest is an ensemble of decision trees.

How Random Forest works

1. Create bootstrap samples from the training rows (sampling with replacement).
2. Train a decision tree on each sample.
3. At each split, consider a random subset of features.
4. Each tree predicts a class for a new car.
5. Combine the tree predictions using majority voting.

Why does randomness help?

It makes trees less alike. If every tree were identical, combining them would add little benefit. Diverse trees can make different errors; voting can reduce variance and make predictions more stable.

Bagging vs Boosting

Bagging	Boosting
Trees/models are generally trained independently on sampled data.	Models are trained sequentially; later models focus on previous errors.
Often reduces variance and stabilizes predictions.	Often focuses on reducing bias and correcting errors.
Random Forest is bagging-style.	Examples: AdaBoost, Gradient Boosting, XGBoost.

Advantages

- Strong baseline for tabular data.
- Captures non-linear patterns and feature interactions.
- Usually more stable than one individual tree.
- Has classifier and regressor variants.

Limitations

- More memory and computation than one small tree.
- Less interpretable than a single shallow tree.
- One-hot encoding may create many columns for high-cardinality features.
- Performance depends on data quality and parameter choices.

6. Evaluation: explain the output

```
print('Accuracy:', accuracy_score(y_test, predictions))
print('Confusion matrix:\\n', confusion_matrix(y_test, predictions))
print(classification_report(y_test, predictions))
display(X_test.head(1))
print('Predicted class:', model.predict(X_test.head(1))[0])
```

Accuracy

Accuracy = correct predictions / total predictions. If 90 out of 100 test examples are correct, accuracy is 90%. It is simple, but may hide poor performance on minority classes.

Confusion matrix

It compares actual and predicted labels. In scikit-learn, rows are actual classes and columns are predicted classes. Diagonal values are correct classifications; off-diagonal values are errors. The default class order is sorted unless labels are supplied explicitly.

Precision, recall, F1 and support

Metric	Meaning	Example interpretation
Precision	Among items predicted as a class, how many truly belong to it?	Of cars predicted “good”, how many are actually good?
Recall	Among all actual items of a class, how many were found?	How many actual “good” cars were identified?
F1-score	Harmonic mean of precision and recall.	Balances precision and recall.
Support	Number of actual test examples in that class.	How many test examples belong to the class?

F1 = 2 × (precision × recall) / (precision + recall). Macro average gives each class equal weight. Weighted average weights classes by their support.

Single-row demonstration

X_test.head(1) selects the first test row. display() shows its features. model.predict(...)[0] predicts its class and [0] selects the first returned prediction. It is a demonstration, not another overall metric.

Why report more than accuracy?

If most cars belong to one class, a model may achieve high accuracy by predicting that class often. Precision, recall and F1 show how it performs for each category.

7. Viva questions: theory cross-questions

Q. What is machine learning?

Answer: A computer learns patterns from examples to make predictions or decisions.

Q. What is supervised learning?

Answer: Learning from labelled examples where the correct output is known. This notebook is supervised.

Q. What is classification?

Answer: Predicting a discrete category or label.

Q. What is a feature?

Answer: An input variable used for prediction.

Q. What is a target?

Answer: The output label the model learns to predict.

Q. What is overfitting?

Answer: Learning training examples too closely, including noise, and performing poorly on new data.

Q. What is underfitting?

Answer: The model is too simple to learn important patterns and performs poorly even on training data.

Q. What is generalization?

Answer: The ability to perform well on new, unseen examples.

Q. Why use a test set?

Answer: To estimate generalization on examples not used for training.

Q. What is data leakage?

Answer: Information that should not be available during training improperly influences the model.

Q. Why stratify=y?

Answer: To preserve approximately similar class proportions in train and test partitions.

Q. Why use random_state?

Answer: To make random splitting/model choices reproducible.

Q. What is One-Hot Encoding?

Answer: Representing each category using its own binary indicator column.

Q. Why not encode low/medium/high as 1/2/3?

Answer: That imposes an assumed numeric spacing. One-hot encoding avoids that assumption.

Q. What is majority voting?

Answer: Each tree votes for a class; the most-voted class is the forest prediction.

Q. Can Random Forest perform regression?

Answer: Yes. RandomForestRegressor predicts numeric values; RandomForestClassifier predicts labels.

Q. What is a bootstrap sample?

Answer: A sample drawn from the dataset with replacement; some rows may repeat and some may be left out.

Q. What is feature randomness?

Answer: Each tree split considers a random subset of features, helping trees become diverse.

8. Viva questions: code and functions

Q. What does pd.read_csv return?

Answer: A pandas DataFrame containing the CSV rows and columns.

Q. Does cars.head() modify the dataset?

Answer: No. It only displays the first five rows by default.

Q. What does drop(columns="class") do?

Answer: Returns a DataFrame without the class column; it does not delete the column from the original cars object.

Q. What is the difference between X and y?

Answer: X contains input features; y contains target labels.

Q. What does test_size=0.2 mean?

Answer: 20% test data and 80% training data, approximately.

Q. What if stratify is removed?

Answer: The random split may produce different class proportions in train and test.

Q. What does OneHotEncoder learn?

Answer: The categories present in each feature when it is fitted on training data.

Q. What does n_estimators=100 mean?

Answer: The forest contains 100 decision trees.

Q. Does random_state=42 improve accuracy?

Answer: No guarantee. It makes randomness repeatable; a different seed may give a different score.

Q. What does model.fit do?

Answer: Fits the encoder and then trains the forest using transformed training features and y_train.

Q. What does model.predict return?

Answer: An array of predicted class labels, one for each input row.

Q. Why use a pipeline?

Answer: It applies the same learned preprocessing consistently at training and prediction time.

Q. What if the CSV path is wrong?

Answer: pandas raises FileNotFoundError. Check the working directory or use the correct path.

Q. Can it predict a target class absent from training?

Answer: No. A classifier predicts among the classes it learned. Unknown input categories are a separate issue handled by the encoder.

Q. What is the purpose of display()?

Answer: In a notebook, it renders an object such as a DataFrame in a readable table format.

Q. Why is [0] used after predict?

Answer: predict returns an array; [0] selects the first prediction because only one row was supplied.

Potential improvements – mention these as future work

- Inspect data with cars.shape, cars.columns, cars.isnull().sum() and cars["class"].value_counts().
- Compare against a single DecisionTreeClassifier.
- Tune max_depth, min_samples_split, max_features and n_estimators using cross-validation.
- Examine feature_importances_ as one way to understand model reliance.
- Save the fitted pipeline for reuse.

These improvements are not implemented in your uploaded notebook; describe them as suggestions, not as completed work.

9. Final revision sheet

30-second project explanation

“My experiment uses a Random Forest Classifier to predict a car evaluation category. I load the CSV using pandas, separate the input features from the target class, and split the data into training and testing sets using an 80:20 ratio with stratification. Since the features are categorical, I use OneHotEncoder. I combine the encoder and Random Forest in a pipeline. The forest contains 100 decision trees. I train it on the training set, predict labels for the test set, and evaluate the results using accuracy, confusion matrix and classification report. Finally, I demonstrate a prediction for one test example.”

Core code to remember

```
cars = pd.read_csv('Car_Evaluation.csv')
X = cars.drop(columns='class')
y = cars['class']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = make_pipeline(
    OneHotEncoder(handle_unknown='ignore'),
    RandomForestClassifier(n_estimators=100, random_state=42)
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Five tricky cross-questions

Q. Are the 100 trees identical?

Answer: No. Bootstrap samples and random feature subsets make them different.

Q. Does One-Hot Encoding use y?

Answer: No. It transforms X only; y is passed to the classifier for supervised training.

Q. Does handle_unknown ignore an unseen target class?

Answer: No. It handles unknown feature categories. The classifier still predicts only learned target classes.

Q. Does high test accuracy guarantee real-world success?

Answer: No. The test data may not represent future data; distribution changes and data quality matter.

Q. Why is Random Forest often better than one tree?

Answer: Combining diverse trees can reduce variance and make predictions more stable, though it is not guaranteed to win on every dataset.

Before your viva

Run the notebook and note the actual accuracy, confusion matrix, class-wise report and sample prediction. Do not memorize invented numbers. Verify the CSV columns and labels from your own file.

Remember the complete flow: load data → separate X and y → split → encode → train → predict → evaluate. If you can explain why each step exists, you can answer most follow-up questions.